TCD3
Test Automation & Continuous Delivery

Unit Testing

S. Wagner

Software Tests

- How can we assure that our software is correct (Verification) and valid (Validation)?

- Formal Verification
  - i.e., prove it mathematically
  - formal verification proves correctness
  - formally verifiable specifications are hard to write
  - complete verification according to a formal specification is very hard (in the best case it is "only" NP-complete)
  - (currently) formal verification is not a mainstream topic in software development
  - only applied in very critical and special cases

- Testing
  - i.e., try it empirically
  - shows that specific cases do not lead to failures
  - testing cannot guarantee that software is correct (i.e., does not contain *any* bugs)
  - effort can be scaled
  - testing has many facets (e.g., functionality, performance, usability, interaction with other systems, etc.)

Software Defects

- A software defect (aka. bug) is a software problem in general
- Different terms describe propagation of a defect
  - **Error:** human action that produces an incorrect result
  - **Fault:** manifestation of an error in a software system
  - **Failure:** inability of the software system to fulfill its requirements
  - **Incident:** symptom of a failure perceived by the user
- Static analysis tries to identify faults by analyzing code (e.g., code reviews, static code analysis, …)
- Software testing tries to spot failures by dynamically executing code (e.g., unit tests, integration tests, …)

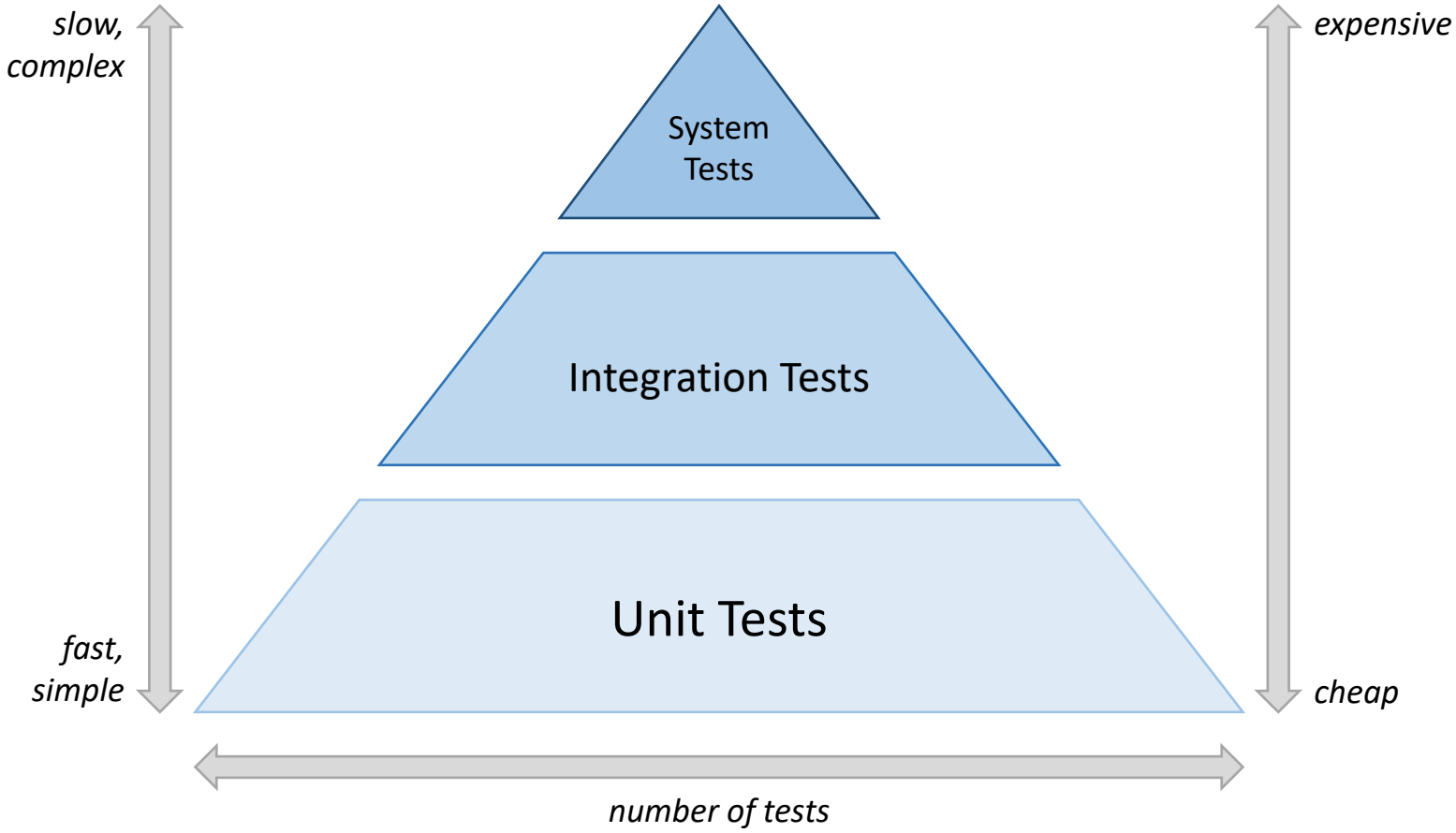
Layers of Software Tests

- Unit Tests = lowest layer
  - checks functionality of a small and clearly separated part (often a single method)
  - mostly white-box tests (tested unit and test are written by the same person)
  - tests are small, often quite simple, and can be executed very quickly
  - easy to automate

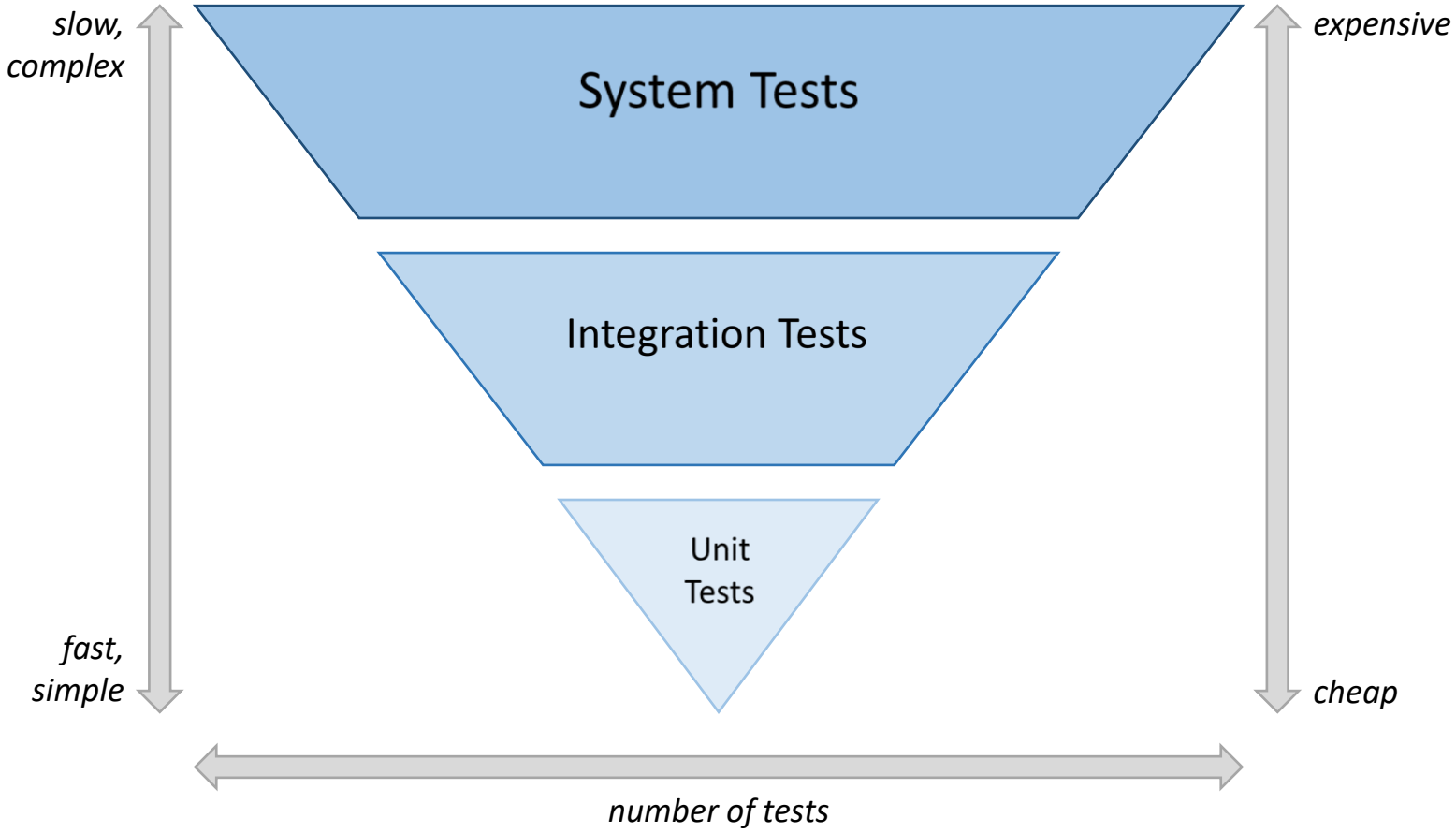
- Integration Tests
  - checks the interaction between different components (for example different classes, subsystems, layers, etc.)
  - can be hierarchically structured (bottom-up vs. top-down)

- System Tests = highest layer
  - system as a whole is tested according to the specification
  - mostly black-box tests (tester and developer are not the same person)
  - often include UI testing
  - tests are comprehensive, complex, and time-consuming
  - hard(er) to automate

Test Pyramid



Test Pyramid in Reality?



Use of Software Tests

- Tests should help to find errors as early as possible (i.e., shortly after they have been introduced)
  - context of the error is still present
  - cause of the error is easier to identify
  - fixing the error does not take long and is cheap
- Developers are responsible for product quality, and not testers
- You cannot test quality into software
- Inverted test pyramid results from manual testing
- Test automation is the key

Purpose of Unit Testing

- Unit tests …
  - lead to a testable software design and thus to higher quality
  - give immediate feedback if something breaks
  - are a safety net when refactoring
  - enable change
  - protect from regressions
  - force us to think about "what" and not about "how"
  - are a form of compilable and executable documentation
  - can be used to explore APIs of new libraries (exploratory testing)

Testability

- Testability describes how easy or hard it is to test a software system
  - software quality criterion which belongs to maintainability (cf. ISO/IEC 25010)
  - bad testability → fewer tests → more failures → higher cost
  - bad testability inverts the test pyramid
  - testability should be kept in mind throughout the whole software lifecycle
- Test-driven development enforces testable software



Impediments for Testability

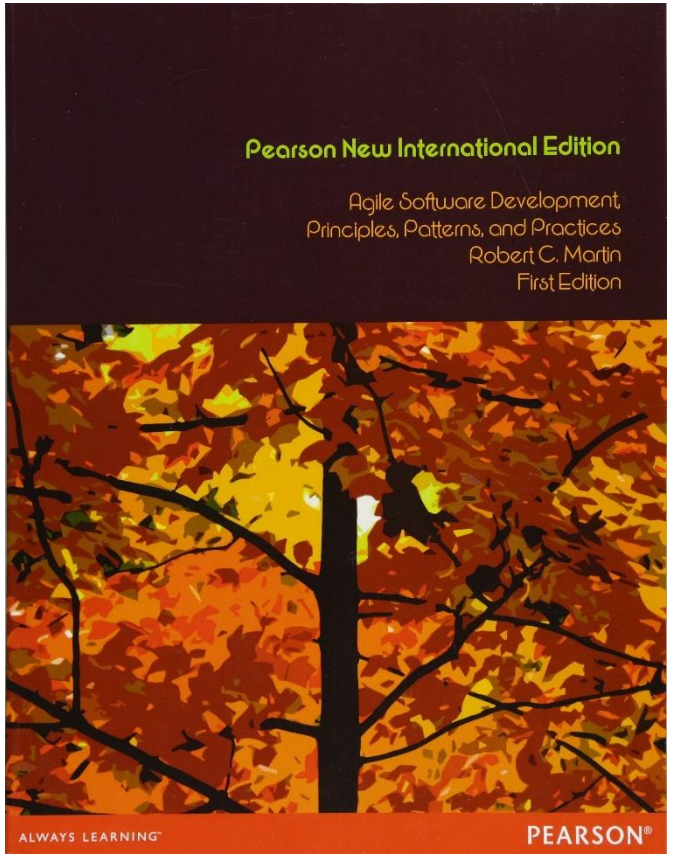
- Testability is bad, if …
  - software entities cannot be isolated
  - coupling between entities is high
  - initialization is inflexible
  - internal states or collaborators cannot be accessed
  - software entities cannot be controlled
  - results are hidden an cannot be observed

- Indicators for Bad Testability
  - writing tests requires much effort
  - automating tests is hard
  - many test doubles are necessary

Principles for Testable Design

… and for good software design in general

S	• Single-Responsibility Principle (SRP)
O	• Open-Closed Principle (OCP)
L	• Liskov Substitution Principle (LSP)
I	• Interface-Segregation Principle (ISP)
D	• Dependency-Inversion Principle (DIP)



Single-Responsibility Principle

A class should have only one reason to change.

- i.e., each software entity should have a single and specific purpose
- leads to better separation of concerns and stronger cohesion
- software entities are simpler and easier to understand
- software entities are only changed on purpose
- sometimes hard to achieve (e.g., in the case of cross-cutting concerns), which motivates other programming paradigms such as aspect-oriented programming (AOP)

- software entities following the SRP are easier to test, because they have a single and clear functionality

Open-Closed Principle

Software entities should be open for extension, but closed for modification.

- introduce reasonable abstractions
- use composition and inheritance for extensions
- clearly define where extension can or cannot take place
- if extensions are necessary, the original code should not change

- abstractions make tests less brittle

Liskov Substitution Principle

Subtypes must be substitutable for their base types.

- derived classes must preserve the basic behavior of their super classes
- otherwise the is-a relationship between classes and their subclasses breaks
- objects of the subclass can replace objects of the super class at any time
- remember that a subclass has to be a specialization
- leads to good abstractions and extensibility

- well separated layers of abstraction are easier to test
- extensible classes can be used as base classes for test doubles

Interface-Segregation Principle

Clients should not be forced to depend on methods that they do not use.

- you should only depend on what you really need
- reduces unnecessary coupling
- makes code much more readable
- related to the SRP

- clear and specific interfaces are easier to test, because they represent a specific purpose

Dependency-Inversion Principle

High-level modules should not depend on low-level modules.
Both should depend on abstractions.

Abstractions should not depend on details.
Details should depend on abstractions.

- program to an interface, not to an implementation
- leads to clearer abstraction and better extensibility
- dependencies should not be hard-coded and should be set as late as possible (cf. dependency injection)

- collaborators can be replaced by test doubles easily

Principles for Good Unit Tests

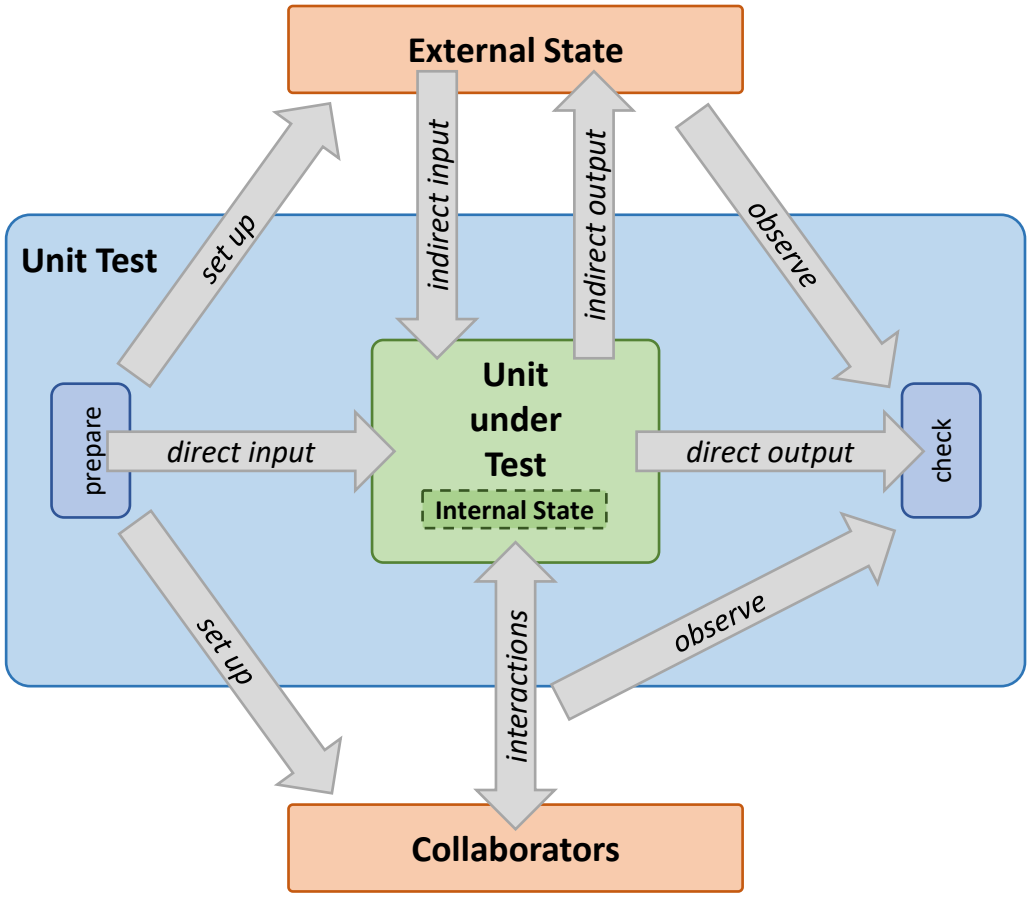
Unit tests should follow the following rules:
(cf. "Clean Code" by Robert Martin)

F	• **fast** … so they are often executed
I	• **independent** … so a failing test does not make other tests fail
R	• **repeatable** … so they can be run in any environment
S	• **self-validating** … so it is clear whether they pass or fail
T	• **timely** … so they will improve the design of production code

Anatomy of a Unit Test

- Direct input and direct output must be prepared and checked
- Internal state has to be initialized and checked
- External state used by the unit under test has to be set up and observed
- External collaborators called by the unit under test have to be set up
- Interactions with external collaborators have to be observed

- AAA Pattern
  - Arrange
  - Act
  - Assert



Arrange – Act – Assert (AAA) Pattern

- Each unit test should consist of three steps:
  - Arrange: prepare the unit under test
  - Act: execute the unit under test
  - Assert: verify the result

```
public class CalculatorTests {
  [Fact]
  public void Add_WithTwoNumbers_ReturnsSum() {
    // arrange
    double a = 1, b = 2, expected = a + b;
    Calculator sut = new();

    // act
    double result = sut.Add(a, b);

    // assert
    Assert.Equal(expected, result);
  }
}
```

- Each unit test should test a single aspect of the system
  - avoid testing multiple test cases in a single unit test
    - if the first assertion fails, all other assertions are not checked anymore
  - each unit test should only contain a single assertion
    - if the test fails, it should be clear what is wrong without having to study the test or its output in detail

Naming of Unit Tests

- Each unit test represents a specific test case
  - if it fails, we want to know what's wrong
  - a detailed inspection of the test should not be necessary
  - in the unit test runner, each test is listed by its name
  - the name must be so descriptive, that we know what caused the failure
- Unit tests (i.e., the test classes and methods) must be named with care
  - some unit test frameworks offer other possibilities to add longer and more descriptive names (cf. @DisplayName in JUnit)
  - otherwise, the method name must contain all required information
    - which unit is tested (what)
    - under which circumstances (when, with)
    - what is the expected result (returns, throws, …)
    - e.g., `GetPackageLoadInfo_WithPackageWhereStatusIsNotOK_ThrowsArgumentException`

DRY vs. DAMP

- DRY = "Don't Repeat Yourself"
  - in production code, code duplication is considered evil
  - common pieces of code should be refactored into methods, classes, etc. so that they can be reused
  - DRY code is easy to maintain and to test, but harder to read

- DAMP = "Descriptive And Meaningful Phrases"
  - code can be read almost like normal text
  - understanding the code is easy and does not require special training
  - good principle for domain specific languages … and unit tests

- Unit tests should not be too DRY and should follow the DAMP principle
  - a unit test stands by itself and is not protected by another test
  - there are no tests to test unit tests
  - simplicity and readability are of major importance for good unit tests
  - fluent APIs are great for unit tests

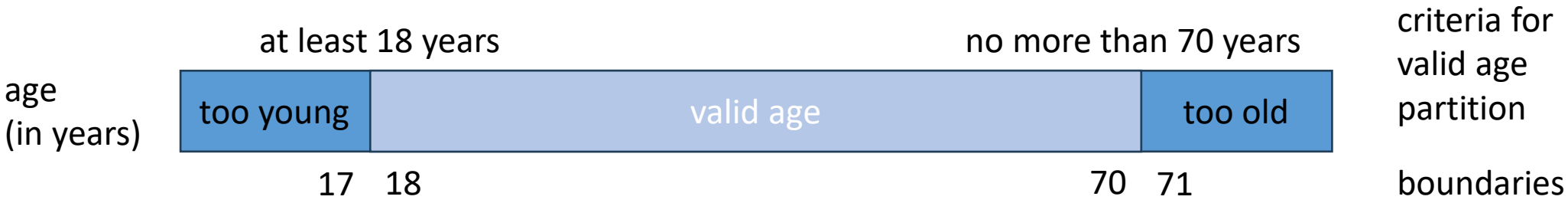
Unit Tests vs. Integration Tests

- Unit tests focus on a single unit
  - they do not involve and test other systems, such as filesystem, network, databases, etc.
  - if the unit under test depends on other units, these other units (collaborators) should be replaced by test doubles
- Integration tests focus on the interplay of different units
  - they test how different units work together, for example a service method and a database
- Border between unit tests and integration tests is often fuzzy

How to Define Reasonable Test Cases?

- Equivalence partitioning
  - think about in which ways the input of a unit test is processed
  - each way represents a specific equivalence class
  - each equivalence class should be tested with a single representative
- Boundary analysis
  - faults often appear at the boundary between different equivalence classes
  - in addition to the chosen representative, these boundary values should be tested as well

Example: Age for donating blood



- Equivalence partitioning
  - Valid equivalence class
    - 18 - 70
  - Invalid equivalence classes
    - < 18
    - > 70

invalid age	valid age	invalid age
e.g., 7, 15	e.g., 25, 57	e.g., 73, 95

- Boundary analysis

invalid age	valid age		invalid age
17	18	70	71

xUnit

https://xunit.net
https://github.com/xunit/xunit



- xUnit is an open-source unit testing framework for .NET
  - authored by Brad Wilson and James Newkirk, who also created NUnit v2
  - xUnit 1.0 was released 5 years after the release of NUnit v2
  - xUnit is very flexible and can be customized in numerous ways
  - samples: https://github.com/xunit/samples.xunit

- Other alternatives:
  - NUnit (https://nunit.org)
  - MSTest
  - see https://learn.microsoft.com/en-us/dotnet/core/testing

Differences between NUnit, MSTest and xUnit

NUnit 3.x	MSTest 15.x	xUnit.net 2.x	Comments
`[Test]`	`[TestMethod]`	`[Fact]`	Marks a test method.
`[TestFixture]`	`[TestClass]`	*n/a*	xUnit.net does not require an attribute for a test class; it looks for all test methods in all public (exported) classes in the assembly.
`Assert.That` `Record.Exception`	`[ExpectedException]`	`Assert.Throws` `Record.Exception`	xUnit.net has done away with the ExpectedException attribute in favor of `Assert.Throws`. See Note 1
`[SetUp]`	`[TestInitialize]`	Constructor	We believe that use of `[SetUp]` is generally bad. However, you can implement a parameterless constructor as a direct replacement. See Note 2
`[TearDown]`	`[TestCleanup]`	`IDisposable.Dispose`	We believe that use of `[TearDown]` is generally bad. However, you can implement `IDisposable.Dispose` as a direct replacement. See Note 2
`[OneTimeSetUp]`	`[ClassInitialize]`	`IClassFixture<T>`	To get per-class fixture setup, implement `IClassFixture<T>` on your test class. See Note 3
`[OneTimeTearDown]`	`[ClassCleanup]`	`IClassFixture<T>`	To get per-class fixture teardown, implement `IClassFixture<T>` on your test class. See Note 3
n/a	*n/a*	`ICollectionFixture<T>`	To get per-collection fixture setup and teardown, implement `ICollectionFixture<T>` on your test collection. See Note 3
`[Ignore("reason")]`	`[Ignore]`	`[Fact(Skip="reason")]`	Set the Skip parameter on the `[Fact]` attribute to temporarily skip a test.
`[Property]`	`[TestProperty]`	`[Trait]`	Set arbitrary metadata on a test
`[Theory]`	`[DataSource]`	`[Theory]` `[XxxData]`	Theory (data-driven test). See Note 4

https://xunit.net/docs/comparisons

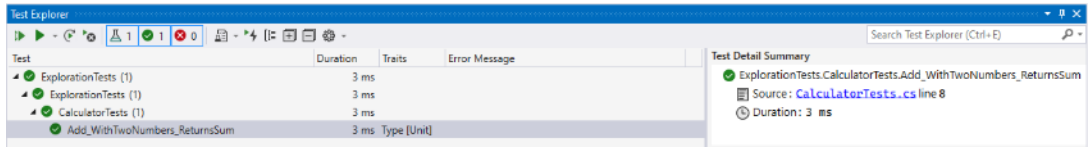
A First Unit Test

- Unit tests are regular methods and classes
  - test methods and classes must be public
- Test methods must be marked with the [Fact] attribute
  - property DisplayName can be used to set a more readable name
  - property Skip can be used to (temporarily ;-) deactivate test methods
- Tests can be categorized with the [Trait] attribute

```
public class CalculatorTests {
  [Fact, Trait("Type", "Unit")]
  public void Add_WithTwoNumbers_ReturnsSum() {
    // arrange
    Calculator sut = new();
    double a = 1, b = 2, expected = a + b;

    // act
    double result = sut.Add(a, b);

    // assert
    Assert.Equal(expected, result);
  }
}
```



- Remember the AAA-Pattern

Assertions

- Assertions are necessary to decide whether the test succeeded or failed
  - expected state is compared with the state of the unit under test
  - expected state comes from a so-called test oracle
    - in the simplest case the test oracle is the person who writes the unit tests and provides the expected state as a literal
    - in more complex cases creating the expected state might require additional logic
- xUnit's assertions are defined as static methods of the Assert class
  - Examples: True, False, NotNull, Equal, NotEqual, Same, NotSame, Throws, Contains …
- Other assertion libraries can also be used
  - if more complex/specific assertions are necessary
  - in order to improve readability
  - Example: Fluent Assertions, https://fluentassertions.com

```
double value = 3.1415927;
value.Should().BeApproximately(3.14, 0.01);
```

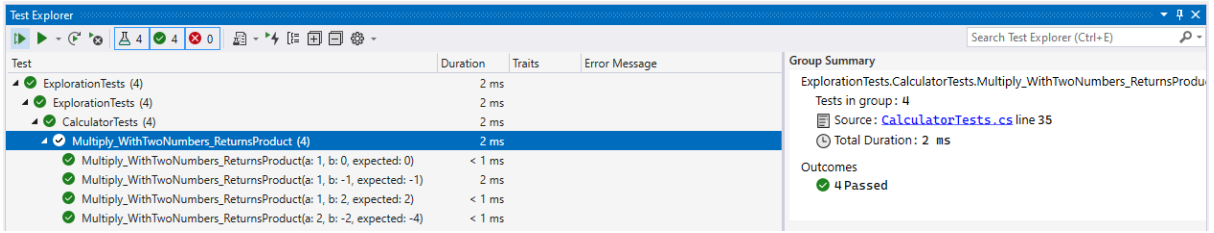
Parameterized Tests

- Parameterized tests enable to run a test multiple times with different arguments
- Parameterized tests are indicated by the [Theory] attribute
- Arguments have to be provided by an attribute
  - [InlineData] provides data as literal values
  - [MemberData] defines a member which is called to provide the data
  - [ClassData] defines a class which is used to provide the data
- Arguments of a single test case are provided as object[]
- […Data] attributes can be combined

```
public class CalculatorTests {
  [Theory]
  [InlineData(1, 2, 2)]
  [InlineData(1, 0, 0)]
  [MemberData(nameof(Multiply_Data))]
  public void Multiply_WithTwoNumbers_ReturnsProduct(double a, double b,
                                                     double expected) {
    Calculator sut = new();

    double result = sut.Multiply(a, b);

    Assert.Equal(expected, result);
  }
  public static IEnumerable<object[]> Multiply_Data() {
    yield return new object[] { 1, -1, -1 };
    yield return new object[] { 2, -2, -4 };
  }
```



Initialization and Fixtures

- Setup and teardown of tests can be done in the constructor and the Dispose method of a test class
  - test class is instantiated for each test case execution
  - data is not shared between test cases, but setup and teardown code is reused
- Test class can implement IClassFixture to share test context between all test cases
  - fixture is injected into the constructor of the test class
  - single fixture instance is reused for all test cases in the class
- ICollectionFixture is used to share test context between different test classes of the same collection

```
public class InitializationDemo : IDisposable {
  private Stack<int> stack;

  public InitializationDemo() {
    stack = new Stack<int>();
  }
  public void Dispose() {
    stack.Clear();
  }

  [Fact]
  public void Test() {
    Assert.NotNull(stack);
  }
}
```

```
public class StackFixture : IDisposable {
  public Stack<int> stack;
  public StackFixture() { stack = new(); }
  public void Dispose() { stack.Clear(); }
}
```

```
public class ClassFixtureDemo : IClassFixture<StackFixture> {
  private readonly StackFixture fixture;

  public ClassFixtureDemo(StackFixture fixture) {
    this.fixture = fixture;
  }

  [Fact]
  public void Test() {
    Assert.NotNull(fixture.stack);
  }
}
```

Test Execution Order

- Test methods are executed in a deterministic, but nonobvious order
- Unit tests should be isolated and independent of each other, therefore the order of execution should not matter
- However, order of execution might be important for integration or acceptance tests
- Custom order in which test methods are executed is provided by a class implementing ITestMethodOrderer
- Custom orderer is defined for a test class by using the [TestMethodOrderer] attribute
- Order of test cases (i.e., test classes) can be defined by implementing the ITestCaseOrderer interface

```
public class AlphabeticalOrderer : ITestMethodOrderer {
  public IReadOnlyCollection<TTestMethod?> OrderTestMethods<TTestMethod>(
                                IReadOnlyCollection<TTestMethod?> testMethods)
                                where TTestMethod : notnull, ITestMethod {
    return testMethods.OrderBy(x => x?.MethodName).ToList();
  }
}
```

```
[TestMethodOrderer(typeof(AlphabeticalOrderer))]
public class TestOrderDemo {
  private static bool testACalled = false;
  private static bool testBCalled = false;

  [Fact]
  public void TestA() {
    testACalled = true;
    Assert.False(testBCalled);
  }

  [Fact]
  public void TestB() {
    testBCalled = true;
    Assert.True(testACalled);
  }
}
```

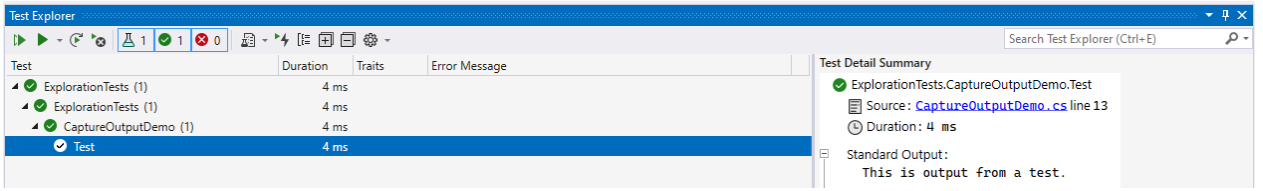
Capture Output from a Test

- ITestOutputHelper enables to capture additional output from a test
  - message is shown in the test output
- instance of ITestOutputHelper is provided by parameter injection to the constructor of test classes

```
public class CaptureOutputDemo {
  private readonly ITestOutputHelper output;

  public CaptureOutputDemo(ITestOutputHelper output) {
    this.output = output;
  }

  [Fact]
  public void Test() {
    output.WriteLine("This is output from a test.");
  }
}
```



JUnit

https://junit.org



- Open-source unit testing framework for Java
  - started in 1995
  - Kent Beck and Erich Gamma developed a first version of JUnit on a flight from Zurich to Washington, D.C. by porting Kent's unit testing framework from Smalltalk to Java
  - according to Martin Fowler: "Never in the field of software development was so much owed by so many to so few lines of code."
- De facto standard for unit testing in Java
- Current Version 6

JUnit



- JUnit 3
  - test cases are defined by inheriting from base class `TestCase`
  - latest version was JUnit 3.8.2 released in 2007
- JUnit 4 (https://github.com/junit-team/junit4)
  - test cases are defined by annotations (Java 5 or later)
  - first version was released in 2006
  - today still heavily used in Java Enterprise domain
  - limited extensibility and modularity
- JUnit 5 (https://github.com/junit-team/junit5)
  - new architecture, execution, and extension model
  - developed to be forward and backward compatible
  - development was seeded with a crowdfunding campaign on Indiegogo in 2015 (€ 53.937 by 474 backers)
  - first version was released in 2017
- JUnit 6 (https://github.com/junit-team/junit-framework)
  - First version released in Sept. 2025
  - requires Java 17+
  - unified versioning for all JUnit projects
  - "old" deprecated modules removed, esp. with extensions new ones are available
  - adoption of JSpecify – annotations to mark nullable types in the API

JUnit



Composed of three sub-projects:

- JUnit Platform
  - foundation for launching testing frameworks on the JVM
  - defines the `TestEngine` API
  - first-class support with popular IDEs
  - concepts
    - Container (= node in the test tree containing other containers or tests, e.g., test class)
    - Test (= node in the test tree that verifies a specific behavior, e.g., `@Test method`)
- JUnit Jupiter
  - programming and extension model for writing JUnit tests and extensions
  - concepts
    - Lifecycle methods: annotated with `@BeforeAll`, `@AfterAll`, `@BeforeEach`, or `@AfterEach`
    - Test Class: class (record) with at least one test method and only a single constructor and must not be abstract
    - Test Method: instance method annotated with `@Test`
- JUnit Vintage (deprecated)
  - test engine for older JUnit versions

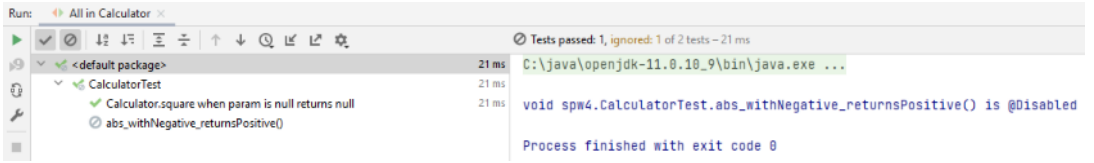
A First Unit Test

- Unit tests are regular methods and classes
- Test methods must be marked with the @Test annotation
- Additional annotations:
  - @DisplayName can be used to set a more readable name
  - @Disabled can be used to (temporarily ;-) deactivate test classes or test methods
  - @Tag can be used to tag and filter tests
- remember the AAA-Pattern

```java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class CalculatorTest {
    @DisplayName("Calculator.square when param is null returns null")
    @Tag("unit")
    @Test
    void square_withNull_returnsNull() {
        Calculator calc = new Calculator();  // arrange
        var result = calc.square(0);         // act
        assertEquals(0, result);             // assert
    }

    @Disabled
    @Test
    void abs_withNegative_returnsPositive() {
        Calculator calc = new Calculator();
        var result = calc.abs(-3);
        assertEquals(3, result);
    }
}
```



Assertions

- Assertions are necessary to decide whether the test succeeded or failed
  - expected state is compared with the state of the unit under test
  - expected state comes from a so-called test oracle
    - in the simplest case the test oracle is the person who writes the unit tests and provides the expected state as a literal
    - in more complex cases creating the expected state might require additional logic
- JUnit's assertions are defined as static methods of the Assertions class
- Examples:
  - assertTrue, assertFalse, assertNotNull, assertEquals, assertSame, assertArrayEquals, assertThrows, …
- Assertions can be grouped with assertAll
  - all assertions of a group are always executed
- Message can be provided for more descriptive errors
  - lazy evaluation of messages is also supported (Supplier<string> message)

- API reference: https://junit.org/junit5/docs/current/api/org.junit.jupiter.api/org/junit/jupiter/api/Assertions.html

Alternative Assertion Libraries

- Other assertion libraries can also be used
  - if more complex/specific assertions are necessary
  - in order to improve readability
- Some examples:
  - Hamcrest
    - provides different matchers
    - http://hamcrest.org
  - AssertJ
    - fluent API for assertions
    - https://assertj.github.io/doc
  - and others …

```
assertThat(1.95, closeTo(2, 0.1));
assertThat("hello hamcrest", is(not(emptyString())));
assertThat("hello hamcrest", both(containsString("hello")).and(containsString("hamcrest")));
assertThat(intArray, arrayContainingInAnyOrder(3, 2, 1));
assertThat(persons, everyItem(hasProperty("age", is(greaterThan(40)))));
```

```
assertThat(1.95).isCloseTo(2, offset(0.1));
assertThat("hello assertj").isNotEmpty();
assertThat("hello assertj").contains("hello").contains("assertj");
assertThat(new Integer[] {1, 2, 3}).containsExactlyInAnyOrder(3, 2, 1);
assertThat(persons).allMatch(person -> person.getAge() > 40);
```

Assumptions

- Assumptions define preconditions for test cases
  - if an assumption fails, the test is skipped
  - a skipped test is not considered to be failed
- Assumptions are defined as static methods of the Assumptions class
- Examples:
  - assumeTrue, assumeFalse, assumingThat
- Message can be provided for more descriptive errors
  - lazy evaluation of messages is also supported (Supplier<string> message)

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;
import static org.junit.jupiter.api.Assumptions.*;

class AssumptionsDemo {
    @Test
    void testOnCIServer() {
        assumeTrue("CI".equals(System.getenv("ENVIRONMENT")));
        // perform test only on CI/CD environment
        assertEquals(42, 40 + 2);
    }

    @Test
    void conditionalTest() {
        assumingThat("DEV".equals(System.getenv("ENVIRONMENT")), () -> {
            // check assertion only on local dev environment
            assertEquals(42, 40 + 2);
        });
        // always check remaining assertions
        assertEquals(42, 21 * 2);
    }
}
```

- API reference: https://junit.org/junit5/docs/current/api/org.junit.jupiter.api/org/junit/jupiter/api/Assumptions.html

Conditional Test Execution

- Test methods can be enabled or disabled according to specific conditions
- Conditions are defined with annotations:
  - @EnabledOnOs
  - @EnabledOnJre, @EnabledForJreRange
  - @EnabledIfSystemProperty
  - @EnabledIfEnvironmentVariable
  - each annotation is also available in @Disabled… form
- Custom conditions can also be used with @EnabledIf
  - logic for a custom annotation has to be implemented in a method

```
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.condition.*;
import static org.junit.jupiter.api.Assertions.*;
import static org.junit.jupiter.api.condition.JRE.*;
import static org.junit.jupiter.api.condition.OS.*;

class ConditionalTestDemo {
    @Test
    @DisabledOnOs(WINDOWS)
    void testA() { assertEquals(42, 21 * 2); }

    @Test
    @EnabledForJreRange(min = JAVA_9, max = JAVA_11)
    void testB() { assertEquals(42, 21 * 2); }

    @Test
    @EnabledIfEnvironmentVariable(named = "ENV", matches = ".*STAGING.*")
    void testC() { assertEquals(42, 21 * 2); }

    @Test
    @EnabledIf("customCondition")
    void testD() { assertEquals(42, 21 * 2); }

    boolean customCondition() {
        boolean enabled = true;
        // custom logic to decide whether
        // test should be enabled or not
        return enabled;
    }
}
```

Custom Annotations

- JUnit annotations can be used as meta-annotations
- Custom annotations can be defined to combine other annotations
- Useful to create a shortcut for
  - a specific set of tags
  - conditions
  - etc.

```
import java.lang.annotation.*;
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.condition.*;
import static org.junit.jupiter.api.Assertions.*;
import static org.junit.jupiter.api.condition.OS.*;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Tag("unit")
@Tag("fast")
@EnabledOnOs(LINUX)
@Test
@interface UnitTestOnLinux { }

class CustomAnnotationDemo {
    @UnitTestOnLinux
    void unitTestA() {
        assertEquals(42, 15 + 27);
    }

    @UnitTestOnLinux
    void unitTestB() {
        assertEquals(42, 20 + 22);
    }
}
```

Setup and Teardown of Fixtures

- Setup and teardown of test fixtures can be done in specifically annotated methods
- Test class is instantiated for each test method
  - "per-method" test instance lifecycle
  - avoids side effects
  - instance variables of the test class can be used safely to pass data between setup, test execution and teardown
  - @BeforeAll and @AfterAll methods must be static
- Test instance lifecycle can be changed to "per-class"
  - by adding @TestInstance(Lifecycle.PER_CLASS) to the test class
  - @BeforeAll and @AfterAll then can be non-static methods
  - be aware that tests should not depend on each other and carefully avoid unwanted side effects

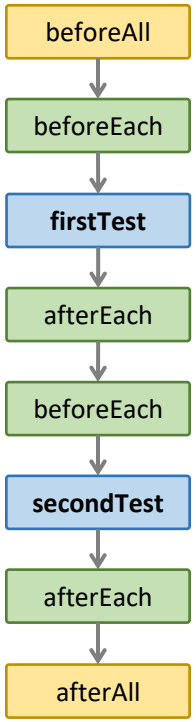
```java
class SetupTeardownDemo {
    @BeforeAll
    static void beforeAll() {
        System.out.println("beforeAll");
    }

    @BeforeEach
    void beforeEach() {
        System.out.println("  beforeEach");
    }

    @Test
    void firstTest() {
        System.out.println("    firstTest");
        assertTrue(true);
    }
    @Test
    void secondTest() {
        System.out.println("    secondTest");
        assertTrue(true);
    }

    @AfterEach
    void afterEach() {
        System.out.println("  afterEach");
    }

    @AfterAll
    static void afterAll() {
        System.out.println("afterAll");
    }
}
```




```
beforeAll
  beforeEach
    firstTest
  afterEach
  beforeEach
    secondTest
  afterEach
afterAll
```

Test Execution Order

- Test methods are executed in a deterministic, but nonobvious order
- Unit tests should be isolated and independent of each other, therefore the order of execution does not matter
- However, order of execution might be important for integration or acceptance tests
- Order in which test methods are execute can be configured with the @TestMethodOrder annotation on the test class
- Built-in MethodOrderer:
  - DisplayName
  - MethodName
  - OrderAnnotation
  - Random

```java
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.MethodOrderer.*;
import static org.junit.jupiter.api.Assertions.*;

@TestMethodOrder(OrderAnnotation.class)
class ExecutionOrderDemo {
    @Test
    @Order(2)
    void testA() {
        System.out.println("testA");
        assertTrue(true);
    }

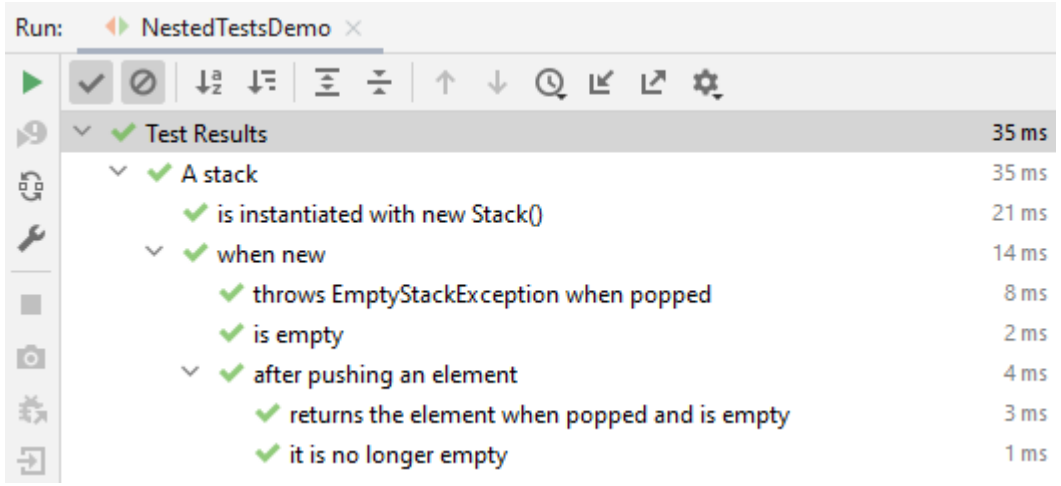
    @Test
    @Order(3)
    void testB() {
        System.out.println("testB");
        assertTrue(true);
    }

    @Test
    @Order(1)
    void testC() {
        System.out.println("testC");
        assertTrue(true);
    }
}
```

```
testC
testA
testB
```

Nested Tests

- Test classes can be nested which enables additional grouping of tests
- Nested tests are implemented as non-static nested classes and must have the @Nested annotation
- Setup and teardown methods of outer tests are always also executed for nested tests
- Hierarchical structure of nested tests is also reflected in the test runner




```java
@DisplayName("A stack")
class NestedTestDemo {
    Stack<String> stack;

    @Test
    @DisplayName("is instantiated with new Stack()")
    void isInstantiatedWithNew() {
        stack = new Stack<>();
    }

    @Nested
    @DisplayName("when new")
    class WhenNew {
        @BeforeEach
        void createNewStack() { stack = new Stack<>(); }

        @Test
        @DisplayName("is empty")
        void isEmpty() {
            assertTrue(stack.isEmpty());
        }

        @Test
        @DisplayName("throws EmptyStackException when popped")
        void throwsExceptionWhenPopped() {
            assertThrows(EmptyStackException.class, stack::pop);
        }

        @Nested
        @DisplayName("after pushing an element")
        class AfterPushing {
            String anElement = "an element";
            @BeforeEach
            void pushAnElement() { stack.push(anElement); }

            @Test
            @DisplayName("it is no longer empty")
            void isNotEmpty() {
                assertFalse(stack.isEmpty());
            }

            @Test
            @DisplayName("returns the element when popped and is empty")
            void returnElementWhenPopped() {
                assertEquals(anElement, stack.pop());
                assertTrue(stack.isEmpty());
            }
        }
    }
}
```

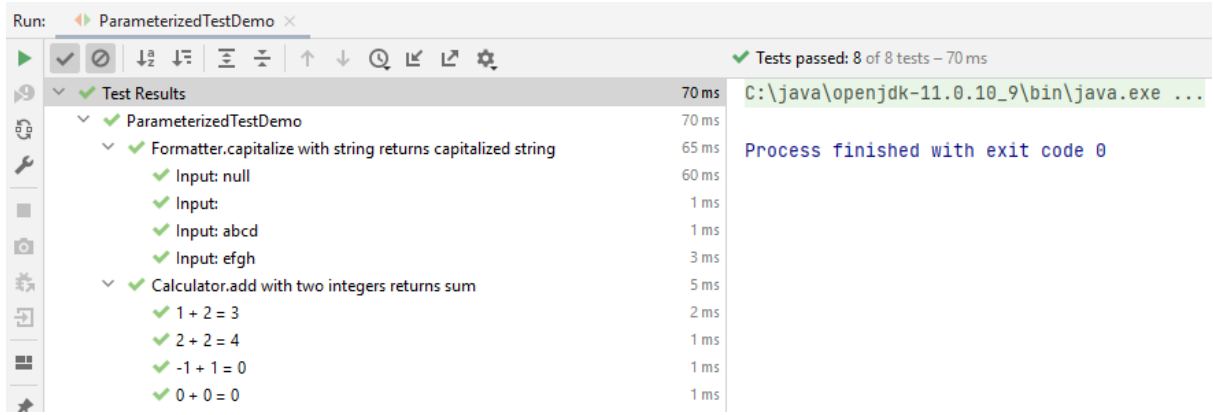
Parameterized Tests

- Parameterized tests enable to run a test multiple times with different arguments
- Arguments have to be provided by a source
  - @NullSource provides null for reference types
  - @EmptySource provides an empty string, list, set, array, etc.
  - @NullAndEmptySource is the combination of the previous two
  - @ValueSource provides single literal values
  - @EnumSource provides values of an enum type
  - @MethodSource defines a method which is called to provide the arguments
  - @CsvSource provides multiple arguments
  - @ArgumentsSource defines a class implementing ArgumentsProvides which is called
- @...Source annotations can be combined
- An individual display name for each execution can be defined
  - placeholders are available for the current arguments

```java
import org.junit.jupiter.api.*;
import org.junit.jupiter.params.*;
import org.junit.jupiter.params.provider.*;
import static org.junit.jupiter.api.Assertions.*;

class ParameterizedTestDemo {
    @DisplayName("Calculator.add with two integers returns sum")
    @ParameterizedTest(name = "{0} + {1} = {2}")
    @CsvSource({"1, 2, 3", "2, 2, 4", "-1, 1, 0","0, 0, 0",})
    void add(int x, int y, int expected) {
        assertEquals(expected, new Calculator().add(x, y));
    }

    @DisplayName("Formatter.capitalize with string returns capitalized string")
    @ParameterizedTest(name = "Input: {0}")
    @NullAndEmptySource
    @ValueSource(strings = {"abcd", "efgh"})
    void capitalize(String s) {
        if (s == null) {
            assertEquals(s, new Formatter().capitalize(s));
        } else {
            assertEquals(s.toUpperCase(), new Formatter().capitalize(s));
        }
    }
}
```

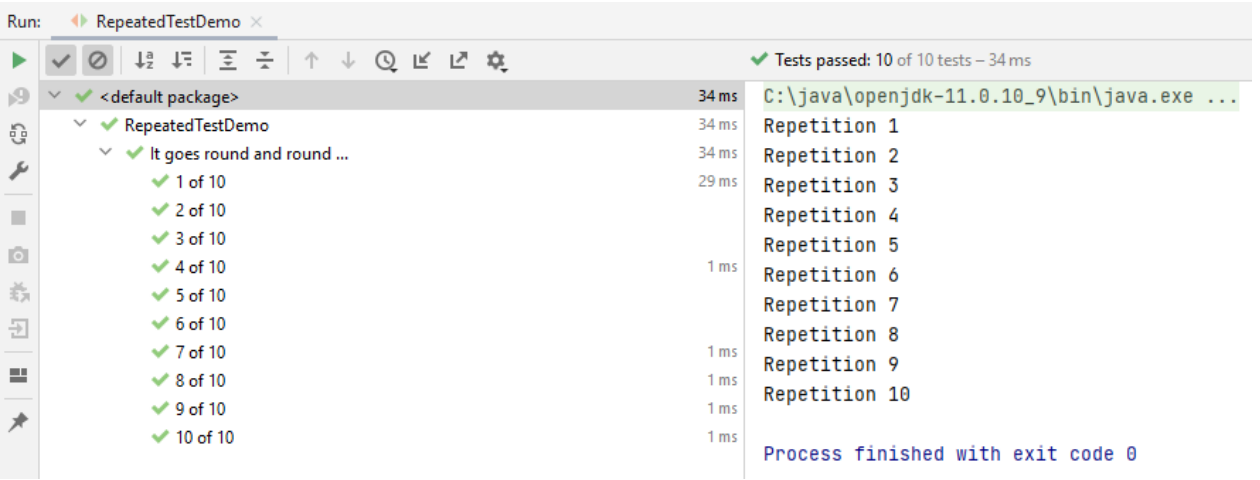


Repeated Tests

- Tests can be repeated for a given number of times
  - for repeated tests the @RepeatedTest annotation has to be used instead of @Test
- Each repetition behaves exactly like a regular test execution
- An individual display name for repetitions can be defined
  - placeholders are available for current repetition, total repetitions, and original display name
- If the test method offers a parameter of type RepetitionInfo, the current repetition and total repetitions can also be used in the test

```java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class RepeatedTestDemo {
    @DisplayName("It goes round and round ... ")
    @RepeatedTest(value = 10, name = "{currentRepetition} of {totalRepetitions}")
    void repeatedTest(RepetitionInfo repetitionInfo) {
        System.out.println("Repetition " + repetitionInfo.getCurrentRepetition());
        assertTrue(true);
    }
}
```

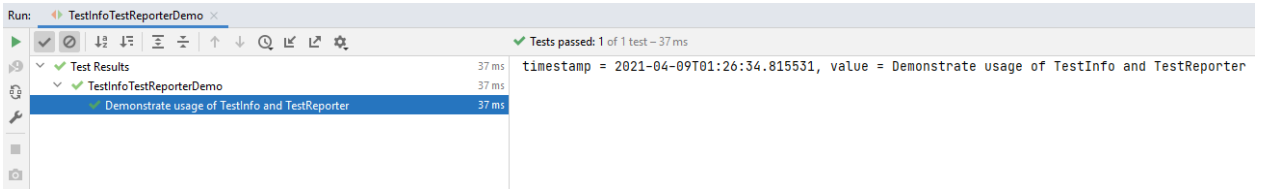


Metadata and Reporting

- TestInfo contains metadata about the current test, i.e.
  - display name
  - test class
  - test method
  - tags
- TestReporter enables to publish additional information about the current test
  - message and timestamp are shown in the test output
- TestInfo and TestReporter are both provided by parameter injection in constructors and methods

```java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class TestInfoTestReporterDemo {
    @Test
    @DisplayName("Demonstrate usage of TestInfo and TestReporter")
    void test(TestInfo testInfo, TestReporter testReporter) {
        testReporter.publishEntry(testInfo.getDisplayName());
        assertEquals(42, 30 + 12);
    }
}
```



Interfaces and Default Methods

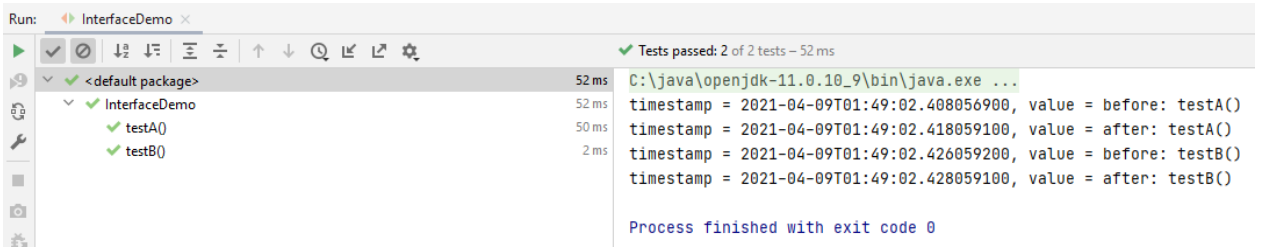
- Test annotations can also be applied on default methods in interfaces
- Common parts (e.g., setup and teardown logic) can be encapsulated
- @BeforeAll and @AfterAll can only be added to default methods, if per-class test instance lifecycle is used

```
interface TestTracer {
    @BeforeEach
    default void traceBefore(TestInfo testInfo, TestReporter testReporter) {
        testReporter.publishEntry("before: " + testInfo.getDisplayName());
    }

    @AfterEach
    default void traceAfter(TestInfo testInfo, TestReporter testReporter) {
        testReporter.publishEntry("after: " + testInfo.getDisplayName());
    }
}

class InterfaceDemo implements TestTracer {
    @Test
    void testA() {
        assertEquals(42, 15 + 27);
    }

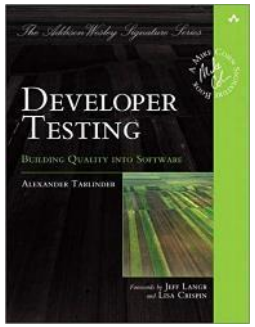
    @Test
    void testB() {
        assertEquals(42, 20 + 22);
    }
}
```



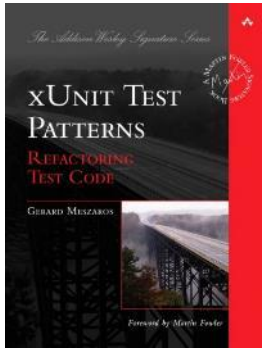
Literature



B. Garcia:
Mastering Software Testing with JUnit 5
Packt Publishing



A. Tarlinder:
Developer Testing: Building Quality into Software
Addison Wesley



G. Meszaros:
xUnit Test Patterns: Refactoring Test Code
Addison Wesley